Advanced Git Practical Command Example

This is a complete hands-on workflow covering: - Creating a repository - Using git stash - Performing merge - Performing rebase

All steps are executable in sequence.

STEP 1: Create Project and Initialize Git

```
mkdir Git_Advanced_Demo        # create project folder
cd Git_Advanced_Demo           # move into folder

git init                       # initialize git repository
```

STEP 2: Create Initial File and Commit

```
touch index.c                  # create file

# add some code manually in index.c

git add index.c                # stage file
git commit -m "Initial commit" # save initial version
```

STEP 3: Create Feature Branch

```
git checkout -b feature        # create and switch to feature branch
```

STEP 4: Make Changes (Unfinished Work)

```
# edit index.c and add some code

git status                     # shows modified file
```

STEP 5: Use STASH (Save Unfinished Work)

```
git stash                       # temporarily save changes
```

```
git status                      # working directory becomes clean
```

STEP 6: Switch to Main and Make Another Change

```
git checkout main               # switch to main branch
```

```
touch main.c                    # create new file

git add main.c
git commit -m "Added main file"
```

STEP 7: Go Back to Feature Branch and Restore Work

```
git checkout feature            # switch back to feature branch

git stash pop                   # restore stashed changes
```

Now your unfinished work is back.

STEP 8: Commit Feature Work

```
git add .
git commit -m "Completed feature work"
```

MERGE WORKFLOW

STEP 9: Merge Feature into Main

```
git checkout main              # go to main branch

git merge feature              # merge feature branch into main
```

This creates a merge commit combining both histories.

REBASE WORKFLOW

STEP 10: Create Another Feature Branch

```
git checkout -b feature2       # new branch from main
```

```
touch feature2.c

git add feature2.c
git commit -m "Feature2 commit"
```

STEP 11: Add New Commit in Main

```
git checkout main

touch update.c

git add update.c
git commit -m "Main update"
```

STEP 12: Rebase Feature2 onto Main

```
git checkout feature2          # switch to feature2

git rebase main                # reapply commits on top of main
```

Now feature2 commits come after latest main commit.

STEP 13: Push After Rebase (if needed)

```
git push origin feature2 --force
```

--force is required because rebase rewrites history.

FINAL UNDERSTANDING

Stash - Temporarily saves unfinished work

Merge - Combines branches and preserves history

Rebase - Moves commits to create clean linear history

If executed step by step, this flow demonstrates real-world usage of stash, merge, and rebase.

Pro.Pranjul Shukla